BỘ GIÁO DỤC VÀ ĐÀO TẠO

TRƯỜNG ĐẠI HỌC SƯ PHẠM KỸ THUẬT HƯNG YÊN

BÁO CÁO ĐỒ ÁN HỌC SÂU

PHÁT HIỆN VẬT CẢN TRONG ẢNH HƯỚNG TỚI HỖ TRỢ DI CHUYỂN CHO NGƯỜI KHIẾM THỊ

So sánh YOLO11n, YOLO11s và Faster R-CNN trên bộ dữ liệu ROD

Sinh viên: **NGUYỄN TRẦN KIÊN**
Mã sinh viên: **12423018**
Mã lớp: **12423TN**
Giảng viên hướng dẫn: **PGS.TS. NGUYỄN MINH TIẾN**

HƯNG YÊN – 2026

NHẬN XÉT

Nhận xét của giảng viên hướng dẫn:

GIẢNG VIÊN HƯỚNG DẪN

PGS.TS. Nguyễn Minh Tiến

LỜI CAM ĐOAN

Tôi cam đoan báo cáo trình bày trung thực công việc và kết quả của dự án phát hiện vật cản. Các số liệu về dữ liệu, huấn luyện, đánh giá và benchmark được đối chiếu với tệp cấu hình, checkpoint hoặc kết quả máy đọc trong kho dự án. Báo cáo phân biệt kết quả đã đo, cấu hình, thông tin tài liệu và nội dung chưa có bằng chứng. Kết quả TEST chưa chạy không được thay bằng suy đoán. Tài liệu tham khảo được ghi nguồn ở cuối báo cáo.

LỜI CẢM ƠN

Tôi xin cảm ơn PGS.TS. Nguyễn Minh Tiến đã hướng dẫn việc xác định bài toán, tổ chức thực nghiệm và trình bày kết quả. Tôi cảm ơn các thầy cô Bộ môn Khoa học máy tính, Khoa Công nghệ thông tin, Trường Đại học Sư phạm Kỹ thuật Hưng Yên đã cung cấp kiến thức nền tảng. Những góp ý về tách dữ liệu, kiểm soát rò rỉ, lựa chọn chỉ số và diễn giải giới hạn giúp báo cáo tránh nhầm lẫn giữa một mô hình có chỉ số tốt và một hệ thống bảo đảm an toàn. Tôi chịu trách nhiệm về các sai sót còn lại.

Tóm tắt

Nghiên cứu khảo sát việc phát hiện vật cản trong ảnh tĩnh với ba mô hình khởi tạo từ trọng số COCO và tinh chỉnh trên ROD Final. Câu hỏi trung tâm là liệu chuyển từ YOLO11n sang YOLO11s có tạo ra mức cải thiện phát hiện đủ lớn so với chi phí tính toán trên phần cứng hạn chế, và Faster R-CNN nằm ở đâu trên đường đánh đổi đó. Bộ dữ liệu đã đóng băng có 25 lớp; số lượng ảnh theo từng split được trình bày trong Chương 2. *[ĐÃ ĐO: F01]*

Trên VALID và cùng bộ đánh giá tự triển khai, YOLO11s đạt Recall cao hơn YOLO11n nhưng chậm hơn trong phép đo CPU; Faster R-CNN đạt chỉ số VALID cao nhất trong ba mô hình nhưng có độ trễ lớn nhất. Đây là kết quả của phép đo cụ thể, chưa chứng minh tính tổng quát trên TEST hoặc khả năng chạy thời gian thực. *[ĐÃ ĐO: F08–F16]* Kết quả TEST cuối đang chờ đánh giá. *[CHƯA CÓ]*

Quy trình nghiên cứu còn đóng góp một chuỗi bằng chứng có thể kiểm toán: manifest làm sạch dữ liệu, lịch sử huấn luyện còn lưu được, checksum checkpoint, cache prediction, cấu hình ngưỡng và khóa đánh giá. Cách tổ chức này cho phép phân biệt rõ kết quả thực đo với cấu hình hoặc giả thuyết. Kết luận thực dụng là E3 phù hợp khi ưu tiên tốc độ CPU, E4 tạo điểm cân bằng cho demo ảnh tĩnh, còn E6 phù hợp khi ưu tiên chất lượng VALID và chấp nhận độ trễ lớn. Mọi lựa chọn vẫn cần xác nhận trên TEST và phần cứng đích.

Mục lục

NHẬN XÉT **1**

LỜI CAM ĐOAN **2**

LỜI CẢM ƠN **3**

Tóm tắt **4**

DANH MỤC CÁC THUẬT NGỮ **10**

1 Bài toán và câu hỏi nghiên cứu **11**

1.1 Mục tiêu và đóng góp 11

1.2 Phạm vi và cấu trúc báo cáo 12

2 Dữ liệu, nhãn và khả năng tái lập **13**

2.1 Taxonomy và phân bố dữ liệu 14

2.2 Chuỗi làm sạch và đóng băng 14

2.3 Nguyên tắc chia tập và chống rò rỉ 15

3 Cơ sở phương pháp **16**

3.1 Hộp bao, mức giao nhau và phát hiện đúng 16

3.2 Precision, Recall, F2 và trung bình độ chính xác 16

3.3 YOLO11n và YOLO11s 17

3.4 Faster R-CNN với ResNet-50 và kim tự tháp đặc trưng 18

3.5 Hàm mất mát và hậu xử lý 18

3.6 Lựa chọn ngưỡng bằng F2 19

3.7 Học chuyển giao và checkpoint 19

3.8 Nguồn sai số và độ bất định . . . 19
3.9 Biểu diễn hộp và hồi quy vị trí . . . 20
3.10 Đường Precision–Recall và AP . . . 20
3.11 Đặc điểm quy mô YOLO11n và YOLO11s . . . 20
3.12 ResNet-50 và kết nối tắt . . . 21
3.13 Feature Pyramid Network . . . 21
3.14 Region Proposal Network và ROI head . . . 22
3.15 Augmentation và khả năng tổng quát . . . 22
3.16 Ý nghĩa của học chuyển giao COCO . . . 22
3.17 Từ detector đến hệ hỗ trợ di chuyển . . . 23

4 Thiết kế thực nghiệm và ứng dụng 24
4.1 Cấu hình huấn luyện . . . 25
4.2 Tính toàn vẹn checkpoint . . . 25
4.3 Giao thức đánh giá chung . . . 26
4.4 Khóa TEST và nguyên tắc lựa chọn . . . 26
4.5 Thiết kế ứng dụng . . . 26

5 Kết quả trên VALID và thảo luận 27
5.1 Chất lượng phát hiện . . . 27
5.2 Chi phí suy luận trên cùng CPU . . . 28
5.3 Chi tiết TP, FP và FN . . . 28
5.4 Kết quả theo lớp . . . 29
5.5 Độ trễ callback và trải nghiệm ứng dụng . . . 30
5.6 Kiểm thử và lựa chọn mô hình . . . 31

6 Giới hạn, phản biện và kết luận 32

A HỒ SƠ TÁI LẬP VÀ HƯỚNG PHÁT TRIỂN 35
A.1 Tái lập đánh giá VALID . . . 35
A.2 Đánh giá TEST cuối . . . 35
A.3 Phân tích lỗi định tính . . . 36

A.4 Đánh giá trên phần cứng mục tiêu 36
A.5 Giới hạn liên quan người dùng khiếm thị 37
A.6 Ánh xạ 25 lớp 37

Danh sách bảng

2.1 Quy mô ROD Final A4 theo split. Nguồn: F01. 13
2.2 Thay đổi quy mô dữ liệu qua các mốc làm sạch 15

4.1 Cấu hình chính của ba thí nghiệm 25

5.1 Kết quả cùng giao thức trên VALID. Nguồn: F09–F11. 27
5.2 Benchmark suy luận cùng CPU; không gồm file I/O và tải model. Nguồn: F14. 28
5.3 AP50–95 theo lớp trên VALID 29
5.4 Độ trễ callback máy chủ trên CPU, 30 ảnh VALID 30

Danh sách hình vẽ

2.1 Phân bố đối tượng theo lớp ở lần phân tích dữ liệu nguồn; số liệu Final lấy từ báo cáo đóng băng. 14

3.1 Luồng đánh giá chung; sơ đồ khái niệm, không phải ảnh kết quả thực nghiệm. 16

3.2 Sơ đồ nguyên lý chung cho hai biến thể YOLO, không thể hiện chi tiết từng lớp mạng. 17

3.3 Sơ đồ hai giai đoạn của Faster R-CNN; RPN được giải thích trong đoạn văn trên. 18

5.1 Đường cong huấn luyện E3 YOLO11n trong 10 epoch đầu của lịch 30 epoch. 29

DANH MỤC CÁC THUẬT NGỮ

Ký hiệu	Cụm từ	Diễn giải
AP	Average Precision	Diện tích nội suy dưới đường Precision–Recall tại một lớp và một ngưỡng IoU.
COCO	Common Objects in Context	Bộ dữ liệu và hệ quy ước đánh giá phát hiện đối tượng phổ biến.
FPN	Feature Pyramid Network	Mạng kim tự tháp đặc trưng dùng biểu diễn nhiều tỉ lệ.
FN/FP/TP	False Negative/False Positive/True Positive	Bỏ sót, báo nhầm và phát hiện đúng theo quy tắc ghép.
IoU	Intersection over Union	Tỷ lệ diện tích giao trên diện tích hợp của hai hộp.
mAP	mean Average Precision	Trung bình AP qua lớp và qua nhiều ngưỡng IoU.
NMS	Non-Maximum Suppression	Hậu xử lý loại các hộp dự đoán trùng lặp.
ROD	Real-Time Obstacle Detection	Bộ dữ liệu vật cản đường phố dùng trong đồ án.
RPN	Region Proposal Network	Mạng đề xuất vùng trong Faster R-CNN.
VALID	Validation split	Tập xác thực dùng chọn checkpoint, ngưỡng và phân tích mô hình.

Chương 1

Bài toán và câu hỏi nghiên cứu

Người sử dụng ảnh cần biết vật thể nào xuất hiện và vật thể đó nằm ở đâu. Bài toán phát hiện đối tượng vì vậy yêu cầu dự đoán đồng thời một hình chữ nhật bao quanh vật thể, tên lớp và điểm tin cậy. Điểm tin cậy là đầu ra của mô hình; nó không phải xác suất an toàn đã được hiệu chuẩn. Từ góc nhìn ứng dụng hỗ trợ, bỏ sót một vật cản quan trọng có thể gây hậu quả hơn một cảnh báo sai, nhưng kết luận về an toàn thực tế đòi hỏi nghiên cứu người dùng và điều kiện sử dụng ngoài phạm vi báo cáo này.

Câu hỏi nghiên cứu là: (i) thay YOLO11n bằng YOLO11s có cải thiện Recall và chất lượng định vị đủ để đánh đổi độ trễ và kích thước hay không; (ii) Faster R-CNN có mang lại lợi ích phát hiện tương ứng với chi phí suy luận hay không; (iii) ba mô hình có thể vận hành trong ứng dụng xử lý ảnh tải lên ở mức độ nào. *[GIẢ THUYẾT]* Mô hình lớn hơn có thể học biểu diễn phong phú hơn, nhưng điều đó không bảo đảm mọi lớp đều tốt hơn hoặc độ trễ phù hợp phần cứng hạn chế. Giả thuyết này được kiểm tra bằng số liệu VALID tại Chương 5, không được coi là kết luận trước khi đo.

Sản phẩm hiện hành chỉ xử lý ảnh tĩnh: tải ảnh, chọn mô hình, phát hiện, hiển thị hộp bao, nhãn, confidence, thống kê và tải ảnh kết quả. *[CONFIG: F17]* Không có webcam, ước lượng khoảng cách, cảnh báo âm thanh, GPS hay dẫn đường; ứng dụng không thay thế phương tiện hỗ trợ di chuyển chuyên dụng.

1.1 Mục tiêu và đóng góp

Mục tiêu thứ nhất là xây dựng bộ dữ liệu sử dụng được cho huấn luyện và đánh giá, giữ đủ 25 lớp nhưng giảm rò rỉ giữa TRAIN, VALID và TEST. Mục tiêu thứ hai là tinh chỉnh ba detector có kiến trúc và quy mô khác nhau trong ngân sách 10 epoch hiện có. Mục tiêu thứ ba là đưa mọi checkpoint qua một evaluator chung để tránh so trực tiếp metric của hai framework. Mục tiêu cuối là tích hợp cả ba vào ứng dụng ảnh tĩnh và đo riêng thời gian suy luận với thời gian

xử lý callback.

Đóng góp của đồ án nằm ở tính truy vết. Mỗi kết luận định lượng có tệp nguồn; checkpoint và ngưỡng có SHA256; TEST được khóa tới bước đánh giá cuối. Cách làm này giảm nguy cơ công bố nhầm số, dùng TEST để tinh chỉnh hoặc quy kết sai nguyên nhân. Báo cáo cũng chỉ ra khoảng trống bằng chứng của E6 và không biến code cấu hình thành lịch sử thực thi đã được kiểm chứng.

1.2 Phạm vi và cấu trúc báo cáo

Đối tượng nghiên cứu là phát hiện đối tượng hai chiều trên ảnh đơn. Phạm vi dữ liệu là ROD Final A4 với 25 lớp. Phạm vi mô hình gồm E3 YOLO11n, E4 YOLO11s và E6 Faster R-CNN ResNet-50 FPN V2, đều khởi tạo từ COCO pretrained. Đánh giá chính dùng VALID; TEST chỉ được dùng một lần sau khi khóa mọi quyết định và hiện chưa chạy.

Báo cáo không đánh giá khoảng cách, quỹ đạo, độ sâu đơn ảnh, giọng nói, định vị, khả năng sử dụng của người khiếm thị hoặc độ an toàn ngoài đời. Các chương sau trình bày dữ liệu, lý thuyết, thiết kế thực nghiệm, kết quả, ứng dụng và giới hạn theo thứ tự đó.

Chương 2

Dữ liệu, nhãn và khả năng tái lập

Nguồn ROD gốc cung cấp ảnh và nhãn phát hiện vật cản trong cảnh đường phố [1]. Dự án sử dụng bản ROD Final A4 đã làm sạch và đóng băng; vì vậy các số liệu của bản Final không được đồng nhất với quy mô dataset card gốc. *[TÀI LIỆU: R1] [ĐÃ ĐO: F01]* Bảng 2.1 nêu số lượng thực tế từ báo cáo freeze; không có ảnh TEST nào được dùng để lựa chọn ngưỡng, mô hình hay thông số huấn luyện trong bản thảo này.

Bảng 2.1: Quy mô ROD Final A4 theo split. Nguồn: F01.

Tập	Ảnh	Đối tượng được gán nhãn
TRAIN	18.151	30.793
VALID	3.318	5.320
TEST	1.611	2.354
Tổng	23.080	38.467

Nhãn được mã hóa bằng chỉ số lớp và hộp bao theo hệ tọa độ YOLO. Vấn đề khi dùng chung hai framework là Torchvision dành chỉ số không cho nền, còn bộ ROD đánh số lớp từ không. Bộ chuyển đổi suy luận ánh xạ nhãn foreground Torchvision sang ROD bằng cách trừ một và bỏ nền, nhờ đó các kết quả đánh giá có chung lớp. *[CONFIG: F03, F06]* Ví dụ, Bike và Motorcycle là hai lớp khác nhau trong bảng nhãn, nên việc phát hiện đúng vị trí nhưng nhầm giữa chúng vẫn là lỗi phân loại. Đây cũng là lý do không thể suy luận chất lượng chỉ từ hình minh họa có hộp bao.

Báo cáo freeze ghi nhận không còn trùng nội dung ảnh chính xác giữa các split và không có lỗi cú pháp hay hình học hộp bao; nó vẫn ghi nhận nhóm trùng trong cùng split và một số nhãn còn chờ duyệt. *[ĐÃ ĐO: F02]* Kiểm tra trùng chính xác không chứng minh mọi ảnh gần trùng đều đã được loại, còn kiểm tra nhãn hợp lệ không chứng minh nhãn đúng về ngữ nghĩa. Do đó, chất lượng đánh giá có giới hạn từ dữ liệu và chưa nên diễn giải một sai khác nhỏ giữa

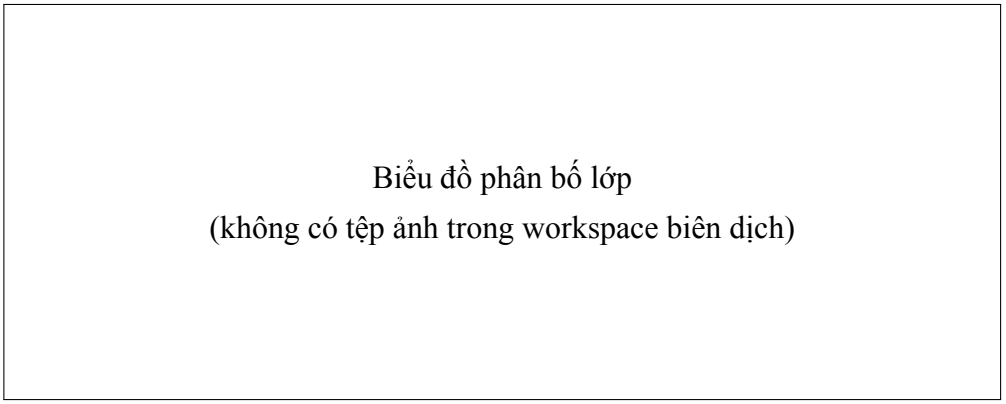
mô hình như tác động chắc chắn ngoài tập VALID.

2.1 Taxonomy và phân bố dữ liệu

ROD Final giữ thứ tự 25 lớp từ ID 0 đến 24: Bike, Building, Car, Person, Stairs, Traffic sign, Electrical Pole, Road, Motorcycle, Dustbin, Dog, Manhole, Tree, Guard rail, Pedestrian crosswalk, Truck, Bus, Bench, Traffic Cone, Fire hydrant, Teraffic Barrel, Plant Pot, Electrical Box, Chair và Bicycle Rack. Chuỗi "Teraffic Barrel" được giữ đúng theo `data.yaml`. Nếu chỉ sửa tên ở một nơi, checkpoint và giao diện vẫn chạy nhưng nhãn hiển thị có thể lệch với evaluator; taxonomy vì vậy là một phần của hợp đồng dữ liệu.

Các lớp bao phủ ba nhóm chính. Nhóm tác nhân chuyển động gồm người, xe đạp, xe máy, ô tô, xe buýt, xe tải và chó. Nhóm cấu trúc đường đi gồm Road, Stairs, Manhole, Guard rail và Pedestrian crosswalk. Nhóm vật thể ven đường gồm biển báo, cột điện, thùng rác, ghế, chậu cây, hộp điện, cọc tiêu, thùng chắn và giá xe đạp. Phân nhóm chỉ hỗ trợ thảo luận; model vẫn học 25 lớp độc lập và không có nhãn cấp cha.

Phân bố lớp không cân bằng. mAP cho mỗi lớp trọng số bằng nhau, trong khi micro Precision/Recall bị chi phối nhiều hơn bởi lớp có nhiều object. Một mô hình có micro Recall cao vẫn có thể bỏ sót lớp hiếm; ngược lại mAP cao không cho biết số cảnh báo sai thực tế tại ngưỡng vận hành. Vì thế báo cáo đọc đồng thời chỉ số tổng, TP/FP/FN và AP theo lớp.



Hình 2.1: Phân bố đối tượng theo lớp ở lần phân tích dữ liệu nguồn; số liệu Final lấy từ báo cáo đóng băng.

2.2 Chuỗi làm sạch và đóng băng

Bản nguồn có 24.326 ảnh và 40.195 object. V2 còn 24.187 ảnh và 40.041 object sau khi loại 139 cặp ảnh–nhãn cùng bốn annotation sai được xác nhận. V3 giữ nguyên quy mô và áp dụng

bảy relabel Bike sang Motorcycle đã được duyệt; 152 đề xuất khác và 18 bbox chưa sửa. V4 cách ly 929 cặp có cạnh leakage trực tiếp, đưa dữ liệu về 23.258 ảnh và 38.797 object. Final cách ly thêm 178 cặp, gồm 146 TRAIN và 32 VALID, không thay đổi TEST.

Bảng 2.2: Thay đổi quy mô dữ liệu qua các mốc làm sạch

Mốc	Ảnh	Object	Thay đổi chính
Nguồn	24.326	40.195	Chuyển đổi nhãn và kiểm toán độc lập.
V2	24.187	40.041	Loại 139 cặp và annotation sai đã xác nhận.
V3	24.187	40.041	Relabel 7 trường hợp Bike sang Motorcycle.
V4	23.258	38.797	Cách ly 929 cặp có leakage trực tiếp.
Final A4	23.080	38.467	Cách ly thêm 178 cặp, giữ nguyên TEST.

Final còn sửa 178 orphan label trong staging sau khi đối chiếu checksum với manifest. Báo cáo xác nhận 0 ảnh thiếu label, 0 label thiếu ảnh, 0 lỗi class ID/cú pháp/hình học và 0 nhóm exact duplicate xuyên split. Tuy vậy còn 271 nhóm exact duplicate trong cùng split, 18 đề xuất bbox và các relabel Bike/Motorcycle chưa duyệt. Chúng là giới hạn dữ liệu, không phải lỗi đã xử lý.

2.3 Nguyên tắc chia tập và chống rò rỉ

TRAIN dùng cập nhật trọng số. VALID dùng theo dõi checkpoint, chọn ngưỡng và so sánh mô hình. TEST chỉ ước lượng cuối sau khi mọi quyết định đã khóa. Nếu ngưỡng được hạ vì một ảnh TEST khó, ảnh đó không còn độc lập và kết quả sẽ lạc quan. Bởi vậy script TEST kiểm hash dữ liệu, checkpoint, ngưỡng và protocol trước khi chạy.

Exact hash phát hiện bản sao byte giống hệt, còn manifest xử lý các quan hệ leakage đã xác định. Nó không chứng minh đã loại mọi near-duplicate do crop, resize hoặc nén lại. Kiểm toán tiếp theo có thể dùng embedding hay perceptual hash với duyệt thủ công; không nên tự động loại ảnh chỉ vì khoảng cách gần nếu chưa xác nhận cùng cảnh.

Chương 3

Cơ sở phương pháp

3.1 Hộp bao, mức giao nhau và phát hiện đúng

Một dự đoán chỉ hữu ích khi vừa đúng lớp vừa phủ đủ vật thể. Với hộp dự đoán B_p và hộp tham chiếu B_g, mức giao nhau trên hợp (intersection over union, IoU) là

$$\mathrm{IoU}(B_p, B_g) = \frac{|B_p \cap B_g|}{|B_p \cup B_g|}. \tag{3.1}$$

Ở đây $|\cdot|$ là diện tích. Bộ đánh giá ghép một dự đoán với tối đa một nhãn thật cùng lớp theo thứ tự điểm giảm dần; dự đoán không ghép được là báo nhầm, nhãn thật không được ghép là bỏ sót. *[CONFIG: F07]* Trên ảnh ROD có hai người sát nhau, một hộp dự đoán phủ cả hai không thể được tính thành hai phát hiện đúng. Nguyên tắc một-một giảm đếm trùng, nhưng kết quả còn phụ thuộc ngưỡng IoU và độ chính xác nhãn gốc.

Ảnh ROD $\longrightarrow$ hộp và điểm tin cậy $\longrightarrow$ ghép cùng lớp theo IoU
$\longrightarrow$ đúng / báo nhầm / bỏ sót $\longrightarrow$ đường cong Precision–Recall

Hình 3.1: Luồng đánh giá chung; sơ đồ khái niệm, không phải ảnh kết quả thực nghiệm.

3.2 Precision, Recall, F2 và trung bình độ chính xác

Gọi TP là số phát hiện đúng, FP là báo nhầm và FN là bỏ sót. Precision $P = TP/(TP+FP)$ phản ánh tỷ lệ dự đoán đáng tin; Recall $R = TP/(TP + FN)$ phản ánh khả năng tìm đủ vật thể. Để ưu tiên Recall khi chọn ngưỡng vận hành trên VALID, dự án dùng

$$F_2 = \frac{(1 + 2^2)PR}{2^2P + R}. \tag{3.2}$$

Hệ số hai là trọng số trong tiêu chí lựa chọn, không biến ứng dụng thành hệ thống an toàn. Ngưỡng confidence của mỗi mô hình được chốt theo micro-F_2 trên VALID và không thay đổi tùy từng ảnh. *[CONFIG: F13]*

Trung bình độ chính xác (average precision, AP) được tính bằng diện tích rời rạc dưới đường cong Precision–Recall sau nội suy. Gọi $p^*(r)$ là Precision tốt nhất tại Recall không nhỏ hơn r; bộ đánh giá dùng

$$AP_t = \frac{1}{101}\sum_{i=0}^{100} p_t^*(i/100), \qquad mAP_{50:95} = \frac{1}{25}\sum_{c=1}^{25}\frac{1}{10}\sum_{t\in\{0.50,0.55,\ldots,0.95\}} AP_{c,t}. \tag{3.3}$$

Chỉ số c là lớp, t là ngưỡng IoU. mAP50 dùng riêng $t = 0.50$. AP quét nhiều mức điểm tin cậy; Precision và Recall trong bảng kết quả lại đo ở một ngưỡng vận hành. Hai loại số này trả lời hai câu hỏi khác nhau. Dự án dùng quy tắc lấy cảm hứng từ COCO [7], nhưng mã chấm riêng không xử lý mọi trường hợp của bộ chấm COCO chính thức, do đó không gọi đây là kết quả COCO benchmark. *[CONFIG: F07]*

3.3 YOLO11n và YOLO11s

YOLO là họ bộ phát hiện một giai đoạn: mạng trích xuất đặc trưng ảnh rồi dự đoán trực tiếp vị trí và lớp trên nhiều tỉ lệ. Các biến thể YOLO11n và YOLO11s dùng cùng họ thiết kế, song quy mô tham số khác nhau [2]. Vấn đề là một vật nhỏ như biển báo có thể mất chi tiết khi giảm kích thước đặc trưng; xử lý đa tỉ lệ cho phép kết hợp thông tin rộng và chi tiết. Trong cấu hình dự án, cả hai nhận ảnh ở cùng kích thước đầu vào, còn trọng số ban đầu lần lượt là `yolo11n.pt` và `yolo11s.pt`. *[CONFIG: F04, F05]*

Ảnh $\longrightarrow$ trích xuất đặc trưng $\longrightarrow$ hợp nhất đa tỉ lệ
$\longrightarrow$ đầu dự đoán hộp / lớp $\longrightarrow$ lọc hộp trùng

Hình 3.2: Sơ đồ nguyên lý chung cho hai biến thể YOLO, không thể hiện chi tiết từng lớp mạng.

Ví dụ ROD gồm người và cầu thang: mạng phải vừa giữ cấu trúc rộng của cầu thang vừa nhận các vùng người nhỏ. Ưu điểm của thiết kế một giai đoạn là đường suy luận gọn; hạn chế tiềm tàng là mô hình nhỏ có ít dung lượng biểu diễn hơn. Đây là nhận định kiến trúc, không phải bằng chứng rằng YOLO11s thắng YOLO11n trên mọi lớp. *[GIẢ THUYẾT]*

3.4 Faster R-CNN với ResNet-50 và kim tự tháp đặc trưng

Faster R-CNN giải bài toán phát hiện theo hai giai đoạn: một mạng đề xuất vùng (region proposal network, RPN) tìm vùng có thể chứa vật thể, rồi đầu xử lý vùng phân loại và tinh chỉnh hộp [3]. ResNet sử dụng kết nối tắt để hỗ trợ học biểu diễn sâu [5]. Kim tự tháp đặc trưng (feature pyramid network, FPN) kết hợp thông tin từ các độ phân giải khác nhau qua nhánh từ trên xuống và nối bên, nhằm xử lý vật thể đa tỉ lệ [4]. Phiên bản V2 trong Torchvision là builder cụ thể được ứng dụng sử dụng [6].

Ảnh $\longrightarrow$ ResNet-50 $\longrightarrow$ FPN
$\longrightarrow$ RPN đề xuất vùng $\longrightarrow$ phân loại / hiệu chỉnh hộp $\longrightarrow$ kết quả

Hình 3.3: Sơ đồ hai giai đoạn của Faster R-CNN; RPN được giải thích trong đoạn văn trên.

Với ảnh ROD có xe ở xa và người ở gần, FPN cung cấp đặc trưng ở nhiều tỉ lệ còn RPN khoanh vùng ứng viên trước khi phân loại. Lợi thế tiềm tàng là quyết định theo vùng có thể phân tích chi tiết; đánh đổi là nhiều phép tính hơn. *[GIẢ THUYẾT]* Cấu hình ứng dụng dùng head 26 đầu ra, gồm nền và 25 lớp ROD; nhãn nền không được hiển thị. *[CONFIG: F06]*

3.5 Hàm mất mát và hậu xử lý

Trong YOLO11, huấn luyện tối ưu đồng thời định vị hộp, phân loại và phân phối khoảng cách biên. Cấu hình E3/E4 đặt hệ số box 7,5; class 0,5 và DFL 1,5. Các hệ số xác định tỷ lệ đóng góp vào gradient, không phải mức quan trọng của lớp trong ứng dụng. Mosaic được bật, biến đổi HSV và lật ngang tạo thêm đa dạng; mixup, cutmix và copy-paste tắt. Hai run dùng cùng thiết lập nên khác biệt n/s được khảo sát trên nền tương đối đồng nhất.

Faster R-CNN trả các loss phân loại, hồi quy hộp, objectness RPN và hồi quy proposal. E6 chuyển nhãn YOLO chuẩn hóa thành hộp pixel `xyxy`, đặt `iscrowd=0`, và cộng một vào class ID vì Torchvision dành 0 cho background. Ảnh qua transform với cạnh ngắn 640, cạnh dài tối đa 960. Đây không phải resize vuông như YOLO, nên so sánh phản ánh pipeline hoàn chỉnh chứ không cô lập tuyệt đối kiến trúc.

Sau forward, hộp score thấp bị lọc và NMS loại dự đoán chồng lặp. NMS quá thấp có thể xóa hai vật thể gần nhau, quá cao có thể giữ nhiều hộp trùng. Evaluator thu prediction YOLO với NMS 0,7; Torchvision có hậu xử lý tích hợp. Đây là một nguyên nhân khiến metric log framework và evaluator chung không trùng hoàn toàn.

3.6 Lựa chọn ngưỡng bằng F2

Prediction được thu một lần tại confidence 0,01, tối đa 300 detection/ảnh, rồi cache được quét qua 95 ngưỡng từ 0,01 đến 0,95. Một prediction là TP nếu cùng lớp, IoU ít nhất 0,5 và ghép với một GT chưa dùng. Khi hai ngưỡng cùng F2, quy tắc ưu tiên Precision rồi ngưỡng cao hơn. Quy tắc tie ghi trước giúp lựa chọn tái lập.

F2 đặt Recall nặng gấp bốn lần Precision trong mẫu số, phù hợp mục tiêu giảm bỏ sót. Nó không làm mọi FN có cùng mức nguy hiểm: bỏ sót bậc thang gần người dùng khác bỏ sót tòa nhà xa, nhưng dữ liệu không có khoảng cách hoặc mức rủi ro. F2 vì vậy là tiêu chí kỹ thuật cho detector, không phải thước đo an toàn di chuyển.

Ngưỡng 0,22/0,19/0,59 khác nhau vì phân phối score của kiến trúc khác nhau. Không thể nói E6 "tự tin hơn" chỉ vì threshold cao. Score chưa được hiệu chuẩn xác suất; 0,8 không có nghĩa đúng 80 phần trăm. Muốn so độ tin cậy cần reliability diagram, expected calibration error hoặc phương pháp calibration phù hợp cho detection.

3.7 Học chuyển giao và checkpoint

Ba mô hình dùng COCO pretrained. E3/E4 nạp `yolo11n.pt`/`yolo11s.pt`; E6 nạp COCO V2 mặc định rồi thay predictor bằng 26 đầu ra. Head mới học 25 lớp ROD và background, còn backbone/FPN bắt đầu từ đặc trưng hình ảnh tổng quát.

Ultralytics lưu best theo fitness ưu tiên mAP50–95. E6 lưu best khi mAP50–95 VALID từ TorchMetrics tăng. Các metric này phục vụ huấn luyện nhưng không đặt chung làm bảng xếp hạng vì code chấm khác nhau. Sau huấn luyện, ba best checkpoint được chấm lại bởi cùng evaluator. Recovery checkpoint giữ optimizer, scheduler, scaler và RNG để resume; best weights đã strip optimizer chỉ dùng suy luận.

3.8 Nguồn sai số và độ bất định

Sai số quan sát gồm sai số mô hình, nhãn, threshold, hậu xử lý và biến thiên theo seed. Một con số mAP bốn chữ số thập phân không có bốn chữ số chắc chắn. Báo cáo chưa bootstrap khoảng tin cậy và chưa chạy nhiều seed, nên chênh lệch rất nhỏ theo lớp chỉ dùng gợi ý kiểm tra.

Trường hợp khó dự kiến gồm vật nhỏ, che khuất, ánh sáng yếu, nền giống vật thể và lớp gần nghĩa như Bike/Motorcycle. Đây là giả thuyết từ đặc điểm detection. Muốn xác nhận phải ghép prediction với ảnh, phân nhóm lỗi theo kích thước/điều kiện và duyệt có quy tắc; bảng FN tự nó không chứng minh nguyên nhân.

3.9 Biểu diễn hộp và hồi quy vị trí

Một hộp có thể biểu diễn bằng hai góc (x_1, y_1, x_2, y_2) theo pixel hoặc bằng tâm, chiều rộng và chiều cao (x_c, y_c, w, h). Nhãn YOLO của ROD chuẩn hóa bốn giá trị sau theo kích thước ảnh. Khi đánh giá, hệ thống chuyển về pixel để tính giao và hợp. Quá trình chuyển phải clip sai số số thực rất nhỏ ở biên nhưng không được chấp nhận hộp rỗng hoặc class ngoài 0–24.

IoU không phản ánh khoảng cách tuyệt đối giữa hai hộp. Với vật nhỏ, lệch vài pixel có thể làm IoU giảm mạnh; với vật lớn, cùng độ lệch có ảnh hưởng nhỏ hơn. Do đó mAP50 đánh giá dễ hơn mAP50–95. Khoảng cách giữa hai chỉ số giúp quan sát chất lượng định vị: model có thể tìm đúng vật thể ở IoU 0,5 nhưng hộp chưa đủ sát để giữ AP ở 0,75–0,95.

Ví dụ một GT có diện tích 10.000 pixel vuông và hộp dự đoán giao 8.000, phần ngoài giao của hai hộp tổng 4.000, khi đó hợp là 12.000 và IoU bằng 0,667. Prediction được tính đúng ở IoU 0,5 nhưng sai ở 0,7. mAP50–95 vì thế khắt khe hơn và được dùng làm tiêu chí best của các pipeline huấn luyện.

3.10 Đường Precision–Recall và AP

Khi hạ confidence threshold, model thường giữ thêm prediction: Recall tăng nhưng FP cũng có thể tăng, làm Precision giảm. Đường Precision–Recall mô tả toàn bộ đánh đổi này. Một điểm vận hành duy nhất phù hợp cho giao diện, còn AP tóm tắt thứ hạng prediction qua nhiều score. Vì thế AP không phụ thuộc đúng một threshold hiển thị, nhưng vẫn phụ thuộc prediction thu thập, NMS và giới hạn detection.

Nội suy 101 điểm lấy Precision cực đại ở Recall không thấp hơn mỗi mức $0,00; 0,01; \ldots; 1,00$. Phép nội suy tránh phạt các dao động cục bộ của đường cong và tạo kết quả tái lập. Nếu một lớp không có GT, cách xử lý phải được quy định; ROD Final có đủ 25 lớp trong toàn bộ dữ liệu và evaluator lưu số GT từng lớp để kiểm tra.

Micro Precision cộng TP, FP, FN của mọi lớp trước khi chia. Macro theo lớp sẽ tính từng lớp rồi trung bình. Dự án dùng micro F2 chọn threshold vì muốn tối ưu số detection tổng thể, còn mAP trung bình đều qua lớp để không bỏ qua lớp hiếm. Hai phép tổng hợp có thể cho thứ hạng khác nhau mà không mâu thuẫn.

3.11 Đặc điểm quy mô YOLO11n và YOLO11s

Ký hiệu n và s thể hiện hai mức quy mô trong cùng họ YOLO11. Checkpoint local sau khi thay head ROD có khoảng 2,595 triệu tham số ở E3 và 9,437 triệu ở E4. Số tham số E4 gấp

khoảng 3,64 lần E3; phép đo công cụ ở ảnh 640 cho 6,53 và 21,72 GFLOPs theo quy ước của Ultralytics/THOP. Các số GFLOPs chỉ so tương đối trong cùng công cụ vì operator và quy ước đếm có thể khác.

Mô hình lớn hơn có thể giữ nhiều kênh đặc trưng và học ranh giới lớp phức tạp hơn. Đổi lại, activation và phép tích chập tăng, làm latency và bộ nhớ cao hơn. Lợi ích không đồng đều theo lớp: E4 tốt hơn E3 ở nhiều lớp nhưng giảm ở Building, Electrical Pole, Motorcycle, Tree, Guard rail và Bicycle Rack trong bảng AP50–95. Một phần chênh lệch nhỏ có thể do biến thiên huấn luyện một seed.

YOLO xử lý toàn ảnh trong một mạng feed-forward và dùng đầu dự đoán dày đặc. Điều này phù hợp khi cần latency thấp. Tuy nhiên, "one-stage" không có nghĩa chỉ một layer, và "real-time" không phải thuộc tính cố định. Tốc độ phụ thuộc ảnh, thiết bị, backend, precision, batch, hậu xử lý và toàn bộ ứng dụng. Báo cáo chỉ gọi E3/E4 nhanh hơn E6 trên CPU đã đo, không khẳng định chuẩn thời gian thực phổ quát.

3.12 ResNet-50 và kết nối tắt

Khi mạng sâu, gradient có thể khó truyền qua nhiều lớp. Residual block học phần dư $F(x)$ rồi cộng đầu vào: $y = F(x) + x$. Đường tắt tạo lối truyền tín hiệu và cho phép block tập trung vào phần thay đổi cần học. ResNet-50 dùng các bottleneck block và tạo nhiều tầng đặc trưng có độ phân giải giảm dần.

Đặc trưng tầng sớm có độ phân giải cao, hữu ích cho biên và vật nhỏ nhưng ít ngữ nghĩa; tầng sâu có ngữ nghĩa mạnh nhưng không gian thô. Nếu chỉ dùng tầng cuối, detector có thể khó định vị vật nhỏ. FPN kết hợp các tầng này để tạo feature map nhiều mức với thông tin ngữ nghĩa mạnh hơn ở mọi tỉ lệ.

3.13 Feature Pyramid Network

FPN có nhánh bottom-up từ backbone, nhánh top-down phóng lớn đặc trưng sâu và lateral connection trộn với đặc trưng cùng kích thước. Kết quả là tập mức pyramid; RPN và ROI head có thể chọn mức phù hợp kích thước vùng. Đối với ROD, người ở xa, biển báo nhỏ và tòa nhà lớn xuất hiện cùng ảnh, nên biểu diễn đa tỉ lệ có lý do kiến trúc rõ ràng.

FPN không bảo đảm tự động tốt hơn YOLO. YOLO cũng có cơ chế tổng hợp đa tỉ lệ trong neck. Khác biệt thực nghiệm còn bao gồm backbone, head, loss, augmentation, input transform và scheduler. Tài liệu FPN giải thích cơ chế, còn kết luận trên ROD phải dựa vào ba checkpoint thực tế.

3.14 Region Proposal Network và ROI head

RPN trượt trên feature map để dự đoán objectness và hiệu chỉnh anchor/proposal. Sau lọc, ROI Align lấy đặc trưng có kích thước cố định từ vùng đề xuất. ROI head phân loại proposal và tinh chỉnh hộp lần nữa. Cách chia hai giai đoạn tập trung tài nguyên vào vùng có khả năng chứa vật thể, nhưng số proposal và xử lý theo vùng làm pipeline nặng hơn.

Trong E6, predictor cuối có 26 lớp gồm background. Khi suy luận, score và label từ Torchvision phải lọc label 0, sau đó trừ một trước khi so với ROD. Nếu quên bước này, mọi tên lớp lệch một vị trí dù hộp đúng. Unit test mapping vì thế quan trọng ngang với test load checkpoint.

RPN và ROI head còn có NMS và số proposal riêng. Benchmark E6 bao gồm tiền xử lý, forward và hậu xử lý theo adapter. Trace FLOPs của mô hình detection có thể bỏ sót ROI/NMS hoặc nhánh điều khiển, nên báo cáo không so trực tiếp ước tính 452 GFLOPs của E6 với GFLOPs YOLO như một đại lượng chính xác tuyệt đối.

3.15 Augmentation và khả năng tổng quát

Augmentation mô phỏng biến thiên không làm đổi nhãn: màu sắc, dịch chuyển, scale và lật ngang. Mosaic giúp mỗi ảnh đầu vào chứa nội dung từ nhiều ảnh nguồn, tăng ngữ cảnh và số object quan sát trong một batch. Lật dọc tắt vì cảnh đường phố lật ngược không thực tế; rotation và shear bằng 0 trong cấu hình hiện tại.

Augmentation quá mạnh có thể tạo cảnh không tự nhiên hoặc hộp rất nhỏ, còn quá yếu dễ overfit. Dự án không chạy ablation augmentation, nên không quy kết chênh lệch E3/E4 cho một phép biến đổi cụ thể. E6 code không có augmentation hình học tùy chỉnh ngoài transform chuẩn; đây là khác biệt giữa pipeline và là biến gây nhiễu khi so kiến trúc.

3.16 Ý nghĩa của học chuyển giao COCO

COCO chứa nhiều lớp phổ biến trùng khái niệm với ROD như person, car, bus, truck và dog. Trọng số pretrained có thể cung cấp cạnh, texture, hình dạng và biểu diễn đối tượng trước khi fine-tune. Các lớp đặc thù hơn như manhole, guard rail hay electrical box cần thích nghi từ dữ liệu ROD.

Pretraining không thay thế đánh giá domain shift. Ảnh ROD có góc chụp, độ phân giải, phân bố lớp và điều kiện cảnh riêng. Ba mô hình đều pretrained nên đồ án không trả lời câu hỏi pretrained so với scratch. E5 scratch thuộc lịch sử và đã ra khỏi phạm vi; đưa E5 không hoàn

chỉnh vào bảng sẽ làm sai thiết kế nghiên cứu hiện hành.

3.17 Từ detector đến hệ hỗ trợ di chuyển

Detector là một thành phần nhận thức, chưa phải hệ điều hướng. Hệ hỗ trợ cần ước lượng khoảng cách hoặc mức nguy cơ, theo dõi vật qua frame, hợp nhất thông tin theo thời gian, chọn cảnh báo và truyền đạt bằng âm thanh/rung. Mỗi bước tạo thêm lỗi và độ trễ. Vì thế Recall detector cao không trực tiếp chuyển thành tỷ lệ tránh va chạm.

Với ảnh tĩnh, giao diện hữu ích cho minh họa cách model nhận cảnh và so checkpoint. Để chuyển sang thiết bị hỗ trợ, yêu cầu phải được xác định cùng người dùng khiếm thị: loại vật cản nào cần cảnh báo, khoảng cách nào có ý nghĩa, tần suất thông báo chấp nhận được và cách xử lý khi model không chắc chắn. Những câu hỏi đó nằm ngoài dữ liệu ROD và cần nghiên cứu riêng.

Chương 4

Thiết kế thực nghiệm và ứng dụng

Ba checkpoint được khởi tạo từ trọng số COCO rồi tinh chỉnh trên cùng ROD Final. E3 và E4 dùng batch, kích thước ảnh, bộ tối ưu, seed và lịch học tương ứng; cả hai chạy 10 epoch đầu của lịch học dự kiến 30 epoch. *[CONFIG: F04, F05]* Không được viết rằng lịch học đã được thiết kế lại cho 10 epoch hoặc rằng mô hình đã hội tụ. Faster R-CNN thuộc framework khác; thiếu log/config local đầy đủ để xác nhận mọi chi tiết huấn luyện giống hai YOLO. *[CONFIG: F06]* Vì vậy, so sánh dưới đây là kết quả giữa các checkpoint đã có trên một giao thức đánh giá, không phải thí nghiệm kiểm soát tuyệt đối mọi khác biệt huấn luyện.

Giao thức chung đọc ground truth ROD Final, chuyển hộp từ tọa độ YOLO chuẩn hóa sang góc hộp theo pixel, ánh xạ lớp Torchvision, thu prediction ở score thấp và sau đó tính AP cũng như chỉ số tại ngưỡng vận hành. Cả ba file kết quả ghi cùng số ảnh/nhãn VALID và cùng bộ tham số chấm. *[CONFIG: F07] [ĐÃ ĐO: F08]* Thứ tự checkpoint và ngưỡng đã lưu trong evaluation lock; khóa cấu hình là điều kiện chuẩn bị cho TEST, không phải bằng chứng TEST đã chạy. *[CONFIG: F18]*

Ứng dụng Gradio đưa ảnh qua adapter của mô hình đã chọn, lọc theo ngưỡng lưu trong cấu hình, vẽ hộp bao và tạo tệp PNG. Model được cache để tránh tải lại sau mỗi lần bấm; thời gian yêu cầu đầu có thể khác rõ rệt thời gian sau khi model đã ở bộ nhớ. *[CONFIG: F17]* Các con số thời gian ở Chương 5 được phân biệt giữa suy luận mô hình và toàn bộ callback máy chủ.

4.1 Cấu hình huấn luyện

Bảng 4.1: Cấu hình chính của ba thí nghiệm

Thuộc tính	E3 YOLO11n	E4 YOLO11s	E6 Faster R-CNN
Khởi tạo	COCO pretrained	COCO pretrained	COCO V2 pretrained
Epoch	10/30 lịch LR	10/30 lịch LR	10 theo code/checkpoint
Batch	8	8	2
Kích thước	640×640	640×640	min 640, max 960
Optimizer	SGD	SGD	SGD
LR ban đầu	0,01	0,01	0,005
Momentum	0,937	0,937	0,9
Weight decay	0,0005	0,0005	0,0005
Scheduler	Ultralytics 30 epoch	Ultralytics 30 epoch	StepLR(3; 0,1)
Seed	0	0	0

E3 và E4 có 10 hàng lịch sử liên tục. Ở epoch 10, learning rate là 0,00703, còn cao hơn đích $lr_0 \times lrf = 0,0001$ của lịch 30 epoch. `close_mosaic=10` chỉ đóng mosaic ở 10 epoch cuối; run dừng trước epoch 20 nên pha này chưa xảy ra. Hai run là 10 epoch đầu của lịch 30, không phải lịch 10 epoch hoàn chỉnh và không chứng minh hội tụ.

E6 dùng AMP, lưu trạng thái giữa epoch và backup định kỳ. Mỗi epoch đánh giá toàn VALID bằng TorchMetrics, bước scheduler rồi cập nhật best. Tuy nhiên `metrics.csv`, `config.json`, `environment.json` và `last_hf_manifest.json` rời thiếu ở local. Code và checkpoint chứng minh kiến trúc nạp được để suy luận; chúng không đủ xác minh độc lập GPU, phiên bản framework và đường cong từng epoch của run.

4.2 Tính toàn vẹn checkpoint

Ba checkpoint có SHA256 đã khóa: E3 `41f794...ed02`, E4 `dc09c7...e2f5`, E6 `ce459c...f9bd`. Adapter từ chối chạy khi hash sai. E6 mang đuôi `.zip` nhưng là state dictionary PyTorch, không phải archive; model được dựng lại bằng builder V2 với head 26 lớp và nạp strict.

Best checkpoint E3 có 5.477.402 byte, E4 19.191.770 byte, E6 173.898.073 byte. Kích thước tệp không bằng bộ nhớ runtime vì activation và workspace phụ thuộc ảnh, framework và thiết bị. Nó vẫn phản ánh chi phí lưu trữ và góp phần vào thời gian request đầu khi model chưa cache.

4.3 Giao thức đánh giá chung

Ground truth YOLO được chuyển từ tâm–rộng–cao chuẩn hóa sang hai góc pixel. Prediction E6 đổi foreground 1–25 về ROD 0–24; background bị bỏ. AP dùng 101 điểm recall tại IoU 0,50 đến 0,95, bước 0,05 và tối đa 100 detection/ảnh. Precision/Recall vận hành ghép tại IoU 0,50 với tối đa 300 detection/ảnh.

ROD không có nhãn crowd/ignore nên evaluator đặt area=all và không bỏ qua crowd. "COCO-style" chỉ mô tả dải IoU và nội suy; implementation không phải pycocotools chính thức. Ba JSON kết quả lưu protocol bên cạnh metric để số đo không bị tách khỏi định nghĩa.

4.4 Khóa TEST và nguyên tắc lựa chọn

Sau VALID, `evaluation_lock.json` ghi hash data YAML, confidence config, checkpoint và protocol. Script TEST yêu cầu khóa hợp lệ cùng cờ xác nhận. Nếu model, threshold hoặc protocol đổi sau khi xem TEST, kết quả không còn độc lập và phải được coi là thí nghiệm khác.

TEST chưa chạy. Giữ trống có chủ đích tốt hơn điền metric từ VALID hoặc log framework. Khi đánh giá cuối, cả ba model cần chạy đúng một lượt trên 1.611 ảnh TEST, không chọn lại ngưỡng và báo metric tổng lẫn theo lớp.

4.5 Thiết kế ứng dụng

Gradio nhận JPG/JPEG/PNG, cho chọn E3/E4/E6, chạy adapter, vẽ bbox và trả PNG cùng danh sách detection. Adapter chuẩn hóa class ID, tọa độ, score và thời gian. Model được cache tối đa ba instance; CUDA được chọn khi có, nếu không dùng CPU. Đường dẫn checkpoint cố định và hash được kiểm trước khi load.

Thời gian suy luận đo riêng quanh model. Thời gian callback bắt đầu từ đối tượng PIL và kết thúc sau kiểm ảnh, inference, vẽ, lưu PNG và tạo văn bản. Nó không gồm upload qua mạng, hàng đợi server và browser render. Tách hai phạm vi đo giúp tránh gọi inference latency là trải nghiệm end-to-end.

Chương 5

Kết quả trên VALID và thảo luận

5.1 Chất lượng phát hiện

Bảng 5.1 sử dụng duy nhất ba tệp kết quả của cùng evaluator trên VALID. Precision và Recall được đo ở ngưỡng riêng đã chọn bằng VALID; AP quét các điểm confidence được thu ở ngưỡng thấp. Việc chọn và báo lại chỉ số trên cùng VALID có thể gây lạc quan, nên TEST cuối độc lập vẫn cần thiết. *[ĐÃ ĐO: F07–F13]*

Bảng 5.1: Kết quả cùng giao thức trên VALID. Nguồn: F09–F11.

Mô hình	Ngưỡng	Precision	Recall	mAP50	mAP50–95	FN
E3 YOLO11n	0,22	0,6455	0,7733	0,7958	0,5951	1.206
E4 YOLO11s	0,19	0,6502	0,8075	0,8049	0,6145	1.024
E6 Faster R-CNN	0,59	0,7483	0,8453	0,8701	0,6604	823

E4 cao hơn E3 3,42 điểm phần trăm Recall và 1,94 điểm mAP50–95; số bỏ sót giảm 182 trên cùng VALID. *[ĐÃ ĐO: F12]* Điều này ủng hộ giả thuyết rằng tăng quy mô từ YOLO11n lên YOLO11s mang lại lợi ích phát hiện trong mẫu đã đo, nhưng không bảo đảm mức tăng tương tự trên ảnh ngoài phân phối. E6 có chỉ số VALID cao hơn E4, song đây chưa phải lời khuyên chọn E6 cho triển khai: cần xét độ trễ, phần cứng và dữ liệu kiểm thử độc lập. *[ĐÃ ĐO: F11, F12]*

Số liệu theo lớp đã có trong các bảng CSV, nhưng bản thảo chưa ghép dự đoán với ảnh lỗi để xác định nguyên nhân bỏ sót, nhầm lớp hay hộp sai ở từng trường hợp. *[ĐÃ ĐO: F19]* Việc diễn giải đặc biệt cho vật cản nhỏ, che khuất hoặc thiếu sáng là *[CHƯA CÓ]* cần phân tầng theo kích thước/điều kiện ảnh và duyệt ví dụ thực tế trước khi kết luận. Nhìn vào một ảnh cầu thang cũng không đủ để kết luận chất lượng chung của checkpoint.

5.2 Chi phí suy luận trên cùng CPU

Độ trễ được đo trên cùng 30 ảnh VALID, một ảnh mỗi lượt, sau các lượt làm nóng. Mức FPS ở đây là nghịch đảo thời gian suy luận trung bình khi xử lý tuần tự, không phải tốc độ webcam hay toàn hệ thống. *[ĐÃ ĐO: F14]* Bảng 5.2 cho thấy E4 dùng nhiều tham số và chậm hơn E3; E6 nặng hơn rõ rệt trên CPU đã đo.

Bảng 5.2: Benchmark suy luận cùng CPU; không gồm file I/O và tải model. Nguồn: F14.

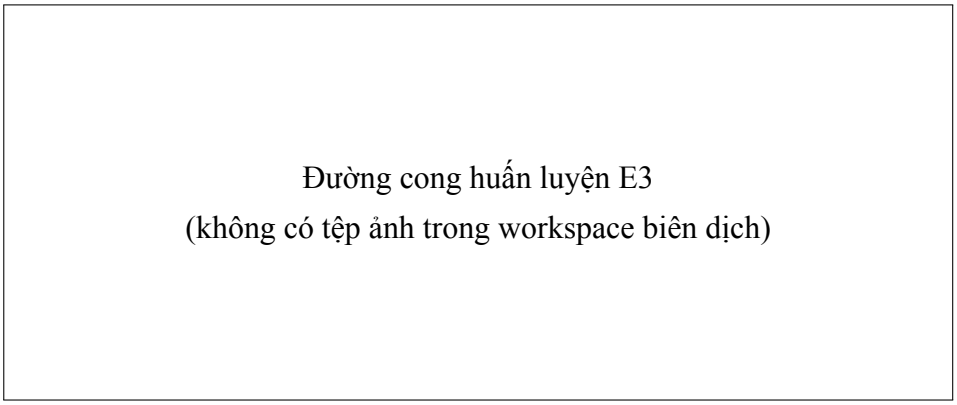
Mô hình	Tham số (triệu)	Mean (ms)	p95 (ms)	FPS tuần tự
E3	2,595	76,26	87,52	13,11
E4	9,437	185,83	360,94	5,38
E6	43,379	2.511,30	3.040,27	0,40

Theo cùng benchmark, E4 có độ trễ trung bình gấp 2,44 lần và số tham số gấp 3,64 lần E3. *[ĐÃ ĐO: F15]* Lợi ích Recall của E4 vì vậy có chi phí cụ thể; lựa chọn phụ thuộc yêu cầu vận hành hơn là một bảng xếp hạng đơn. E6 đạt Recall VALID cao nhất, nhưng độ trễ trung bình CPU gấp 13,51 lần E4. *[ĐÃ ĐO: F11, F15]* Kết quả này chỉ áp dụng cho CPU và phương pháp đo đã ghi, không dự đoán tốc độ GPU hay thiết bị di động.

Benchmark callback tính thêm xác thực ảnh, vẽ hộp, lưu PNG và tổng hợp văn bản. Thời gian trung bình sau warmup lần lượt là 134,13 ms cho E3, 232,63 ms cho E4 và 2.535,22 ms cho E6; request đầu dài hơn do tải model và thiết lập. *[ĐÃ ĐO: F16]* Callback vẫn chưa bao gồm thời gian tải ảnh qua mạng, dựng giao diện trình duyệt hay tương tác con người. Do đó, câu hỏi "có real-time hay không" hiện *[CHƯA CÓ]* cần định nghĩa mục tiêu độ trễ, đo end-to-end trên phần cứng triển khai và với luồng video nếu phạm vi sản phẩm thay đổi. Chỉ số VRAM cũng chưa đo được trong benchmark CPU. *[ĐÃ ĐO: F14]*

5.3 Chi tiết TP, FP và FN

Tại ngưỡng vận hành, E3 có 4.114 TP, 2.259 FP và 1.206 FN; E4 có 4.296 TP, 2.311 FP và 1.024 FN; E6 có 4.497 TP, 1.513 FP và 823 FN. E4 tìm thêm 182 GT so với E3 nhưng phát sinh thêm 52 FP. E6 vừa giảm FN 201 vừa giảm FP 798 so với E4 trên VALID. Vì threshold được chọn bằng F2 trên cùng tập, lợi ích này phải được xác nhận trên TEST.



Hình 5.1: Đường cong huấn luyện E3 YOLO11n trong 10 epoch đầu của lịch 30 epoch.

Log E3 epoch 10 ghi Precision 0,84610, Recall 0,73533 và mAP50–95 0,60605. Số khác bảng evaluator chung vì threshold, NMS, max detection và tích phân AP khác nhau. Báo cáo không trộn hai protocol trong một bảng xếp hạng; log chỉ dùng đọc xu hướng huấn luyện.

5.4 Kết quả theo lớp

Bảng 5.3: AP50–95 theo lớp trên VALID

Lớp	E3	E4	E6
Bike	0,618	0,650	0,782
Building	0,652	0,646	0,635
Car	0,571	0,593	0,619
Person	0,445	0,473	0,539
Stairs	0,529	0,582	0,614
Traffic sign	0,293	0,364	0,380
Electrical Pole	0,451	0,433	0,503
Road	0,407	0,434	0,513
Motorcycle	0,459	0,457	0,439
Dustbin	0,752	0,758	0,769
Dog	0,677	0,747	0,768
Manhole	0,772	0,808	0,803
Tree	0,517	0,490	0,549
Guard rail	0,410	0,373	0,520
Pedestrian crosswalk	0,314	0,345	0,502
Truck	0,567	0,578	0,657

Lớp	E3	E4	E6
Bus	0,593	0,627	0,658
Bench	0,624	0,697	0,710
Traffic Cone	0,634	0,647	0,643
Fire hydrant	0,863	0,906	0,847
Teraffic Barrel	0,849	0,844	0,896
Plant Pot	0,702	0,720	0,743
Electrical Box	0,868	0,884	0,931
Chair	0,671	0,715	0,715
Bicycle Rack	0,640	0,592	0,775

Traffic sign thấp nhất ở cả ba model, còn Electrical Box, Teraffic Barrel và Fire hydrant thuộc nhóm cao. Person và Pedestrian crosswalk quan trọng trong cảnh di chuyển nhưng AP thấp hơn nhiều lớp cố định. E6 cải thiện mạnh Pedestrian crosswalk và Bicycle Rack so với E4, nhưng Motorcycle giảm. mAP tổng cao hơn không đồng nghĩa tốt hơn ở mọi lớp.

Số GT theo lớp khác nhau, nên AP của lớp hiếm có độ bất định lớn. Báo cáo chưa bootstrap khoảng tin cậy và chưa lặp theo seed. Chênh lệch rất nhỏ, như Manhole giữa E4 và E6, không nên diễn giải thành ưu thế chắc chắn. Bảng lớp dùng để ưu tiên kiểm tra ảnh lỗi và thiết kế dữ liệu bổ sung.

5.5 Độ trễ callback và trải nghiệm ứng dụng

Bảng 5.4: Độ trễ callback máy chủ trên CPU, 30 ảnh VALID

Mô hình	Request đầu (ms)	Trung bình warm (ms)	p95 (ms)
E3	975,35	134,13	209,83
E4	653,88	232,63	301,20
E6	5.061,34	2.535,22	2.692,17

Request đầu E4 thấp hơn E3 trong mẫu không có nghĩa E4 luôn tải nhanh hơn; cache hệ điều hành và thứ tự chạy có thể ảnh hưởng. Mean/p95 sau warmup phù hợp hơn để mô tả vận hành lặp lại. Callback gần inference ở E6 vì forward chiếm phần lớn thời gian, còn E3/E4 cho thấy chi phí vẽ và lưu ảnh.

Ngưỡng tự động là 0,22, 0,19 và 0,59; F2 VALID là 0,7439, 0,7703 và 0,8239. Không có slider giúp kết quả nhất quán, nhưng một ngưỡng cho mỗi model không tối ưu riêng từng

lớp. Per-class threshold mới chỉ được khám phá và chưa áp dụng để tránh tăng độ phức tạp khi chưa đánh giá ổn định.

5.6 Kiểm thử và lựa chọn mô hình

Báo cáo ứng dụng ghi một lượt 65 tests và 7 subtests, 0 lỗi. Test bao phủ load checkpoint, checksum, mapping lớp, bbox, no-detection, callback, PNG, CPU fallback và khóa TEST. Đây là kết quả lượt test đã lưu, không chứng minh detector đúng ngoài dữ liệu hoặc trạng thái vừa chạy trong phiên viết.

Nếu ưu tiên tài nguyên và phản hồi nhanh trên CPU, E3 có lợi thế. Nếu cần điểm cân bằng cho demo ảnh tĩnh, E4 tăng Recall và mAP so với E3 trong khi callback trung bình khoảng 0,23 giây. Nếu ưu tiên giảm bỏ sót trên ảnh đơn và chấp nhận khoảng 2,5 giây CPU, E6 là ứng viên. Khuyến nghị phụ thuộc VALID và benchmark local; chưa áp dụng cho smartphone, GPU hay hỗ trợ thời gian thực.

Chương 6

Giới hạn, phản biện và kết luận

Điểm mạnh của so sánh hiện tại là ba mô hình được chấm trên cùng ROD Final VALID và cùng một mã đánh giá, thay vì đặt cạnh nhau các metric do framework khác nhau xuất ra. *[CONFIG: F07]* Tuy nhiên, ngưỡng vận hành được chọn trên VALID và chỉ số tại ngưỡng ấy cũng báo trên VALID; bản thảo chưa có khoảng tin cậy hoặc lặp lại theo seed. Báo cáo freeze còn ghi các trường hợp nhãn chưa được xử lý, và kiểm tra exact duplicate không giải quyết toàn bộ câu hỏi near-duplicate. *[ĐÃ ĐO: F02]* Các yếu tố đó cần nằm trong diễn giải kết quả.

Một phản biện khác là ngân sách huấn luyện ngắn. E3 và E4 chạy 10 epoch đầu của lịch LR 30 epoch, nên kết quả mô tả hiệu năng ở mốc đó chứ không chứng minh hội tụ hoặc ưu thế sau huấn luyện đầy đủ. *[CONFIG: F04, F05]* Với E6, checkpoint local cho phép chạy đánh giá nhưng thiếu bộ log Kaggle đầy đủ để đối chiếu cấu hình và thời gian huấn luyện như hai YOLO. *[CONFIG: F06]* Bảng thời gian huấn luyện công bằng cho ba mô hình vì vậy *[CHƯA CÓ]* cần bổ sung log chính thức trước khi tính chi phí đào tạo.

Trên VALID, YOLO11s cải thiện khả năng phát hiện so với YOLO11n với chi phí CPU lớn hơn; Faster R-CNN cho chất lượng VALID cao hơn nhưng chậm đáng kể ở môi trường đo hiện có. *[ĐÃ ĐO: F09–F15]* Nếu ưu tiên ảnh tĩnh và chất lượng phát hiện, E6 là ứng viên đáng xem xét; nếu ưu tiên độ trễ trên CPU hạn chế, E3 là ứng viên phù hợp hơn để khảo sát. Đây là nhận định có điều kiện, chưa phải lựa chọn sản phẩm cuối. *[GIẢ THUYẾT]* Quyết định cuối cần quy định ngân sách độ trễ, phân tích lỗi theo lớp và đánh giá TEST theo giao thức đã khóa.

Các phép đo còn thiếu

Kết quả TEST cuối: *[CHƯA CÓ]* thực hiện một lượt đánh giá sau khi khóa checkpoint và mọi quy tắc bằng VALID, rồi ghi đầy đủ Precision, Recall, AP từng lớp và lỗi định tính. Thời gian huấn luyện so sánh được, VRAM/GPU và end-to-end trên máy mục tiêu: *[CHƯA CÓ]* lấy từ log cùng tiêu chuẩn đo, không suy từ benchmark CPU. Các điều kiện thiếu sáng và sử dụng

thực tế bởi người khiếm thị: *[CHƯA CÓ]* cần bộ mẫu và thiết kế thử nghiệm chuyên biệt. Những phép đo này không được tự thay bằng kết quả VALID.

Tài liệu tham khảo

[1] Abtinzandi, *ROD-Dataset: Real-Time Obstacle Detection for Smartphone-Based Assistive Vision*, dataset card. https://huggingface.co/datasets/Abtinzandi/Obstacle-Detection-Dataset-YOLO/blob/main/README.md.

[2] Ultralytics, *YOLO11 Model Documentation*. https://docs.ultralytics.com/models/yolo11/.

[3] S. Ren, K. He, R. Girshick, J. Sun, *Faster R-CNN: Towards Real-Time Object Detection with Region Proposal Networks*, NeurIPS, 2015. https://arxiv.org/abs/1506.01497.

[4] T.-Y. Lin, P. Dollár, R. Girshick, K. He, B. Hariharan, S. Belongie, *Feature Pyramid Networks for Object Detection*, CVPR, 2017. https://openaccess.thecvf.com/content_cvpr_2017/html/Lin_Feature_Pyramid_Networks_CVPR_2017_paper.html.

[5] K. He, X. Zhang, S. Ren, J. Sun, *Deep Residual Learning for Image Recognition*, CVPR, 2016. https://openaccess.thecvf.com/content_cvpr_2016/html/He_Deep_Residual_Learning_CVPR_2016_paper.html.

[6] Torchvision, *fasterrcnn_resnet50_fpn_v2 documentation*. https://docs.pytorch.org/vision/master/models/generated/torchvision.models.detection.fasterrcnn_resnet50_fpn_v2.html.

[7] COCO, *Detection Evaluation*. https://cocodataset.org/dataset/detection-eval.htm.

Phụ lục A

HỒ SƠ TÁI LẬP VÀ HƯỚNG PHÁT TRIỂN

A.1 Tái lập đánh giá VALID

Quy trình bắt đầu bằng kiểm tra ROD Final A4 có 18.151 ảnh TRAIN, 3.318 ảnh VALID, 1.611 ảnh TEST và hash `data.yaml` khớp khóa. Tiếp theo kiểm SHA256 ba checkpoint. Không dùng checkpoint cùng tên nhưng khác hash, vì chỉ một thay đổi trọng số cũng tạo thí nghiệm khác.

E3/E4 nhận ảnh ở `imgsz=640`, thu prediction tại confidence 0,01, NMS 0,7 và `max_det=300`. E6 dựng Faster R-CNN ResNet-50 FPN V2, thay head 26 lớp, nạp state dict strict và ánh xạ label 1–25 về 0–24. Prediction được lưu theo image ID để tính lại threshold mà không chạy model lần nữa.

Evaluator kiểm không thiếu/trùng ID, rồi tính AP từng lớp tại mười ngưỡng IoU. Prediction được sắp score giảm dần và ghép greedy với GT cùng lớp chưa dùng. Đường Precision–Recall nội suy 101 điểm. mAP50 trung bình 25 lớp tại IoU 0,5; mAP50–95 trung bình thêm qua mười ngưỡng IoU. Precision/Recall vận hành dùng ngưỡng F2 khóa trước và IoU 0,5.

Khi công bố kết quả cần lưu JSON tổng, CSV theo lớp, hash cache prediction, checkpoint, data YAML và protocol. Nếu chỉ lưu ảnh chụp hoặc một số mAP, không thể xác định dữ liệu, ngưỡng hay code đã dùng. Báo cáo ưu tiên tệp máy đọc vì lý do này.

A.2 Đánh giá TEST cuối

Trước khi mở TEST, kiến trúc, checkpoint, confidence threshold, NMS, max detection, mapping lớp và cách tính metric phải đóng băng. `evaluation_lock.json` là biên bản kỹ thuật. Script phải từ chối nếu hash không khớp.

Ba model sau đó chạy trên đúng 1.611 ảnh TEST. Ngưỡng E3/E4/E6 vẫn là 0,22/0,19/0,59; không điều chỉnh theo TEST. Cần báo Precision, Recall, F2, mAP50, mAP50–95, TP/FP/FN và bảng 25 lớp. Nếu model thắng VALID nhưng không thắng TEST, phải giữ cả hai kết quả và phân tích overfit lựa chọn.

Một lượt TEST không cung cấp biến thiên theo seed. Có thể bootstrap theo ảnh để ước lượng độ bất định chênh lệch metric, nhưng phải giữ nguyên prediction và lấy mẫu ảnh với hoàn lại. Nếu huấn luyện nhiều seed, từng seed cần checkpoint và log riêng; không chọn seed tốt nhất bằng TEST.

A.3 Phân tích lỗi định tính

Mỗi FN nên được gắn thuộc tính: vật nhỏ, che khuất, cắt biên, ánh sáng yếu, motion blur, nền phức tạp, lớp gần nghĩa hoặc nhãn nghi vấn. FP được phân thành nhầm lớp, trùng hộp, nhận nền thành vật thể hoặc phát hiện vật thể thật nhưng thiếu nhãn. Hộp đúng lớp nhưng IoU thấp được tách khỏi lỗi phân loại. Phân loại lỗi cho biết nên cải thiện dữ liệu, augmentation hay kiến trúc.

Mẫu lỗi phải lấy theo quy tắc thay vì chọn ví dụ gây ấn tượng. Có thể lấy ngẫu nhiên theo lớp và loại lỗi, đồng thời báo cỡ mẫu. Với Person, Car và Pedestrian crosswalk có nhiều FN tuyệt đối, cần xem thêm tỷ lệ trên số GT. Với lớp hiếm, vài lỗi có thể làm Recall thay đổi lớn.

Sai nhãn phải ghi vào manifest review và không sửa trực tiếp trong TEST sau khi xem prediction. Nếu xác nhận TEST có lỗi annotation, nên công bố cả kết quả trên bản đóng băng và phiên bản mới cho nghiên cứu sau; không âm thầm sửa để cải thiện model.

A.4 Đánh giá trên phần cứng mục tiêu

Benchmark CPU local 30 ảnh, batch 1 cho phép so tương đối nhưng chưa đại diện smartphone hoặc GPU. Đánh giá triển khai phải cố định thiết bị, driver/framework, chế độ nguồn, kích thước ảnh, số luồng và warmup; báo mean, median, p95, request đầu, bộ nhớ đỉnh và nhiệt độ nếu chạy dài.

Nếu chuyển sang video, FPS ảnh tĩnh chưa đủ. Hệ thống cần đo camera capture, resize, inference, tracking, vẽ, âm thanh và hàng đợi; khi xử lý chậm hơn camera phải quy định bỏ frame. Độ trễ cảnh báo quan trọng hơn throughput trung bình vì frame chậm có thể tạo khoảng im lặng nguy hiểm.

Tối ưu có thể gồm ONNX, TensorRT, TorchScript, lượng tử hóa hoặc giảm kích thước

ảnh. Mỗi thay đổi phải đánh giá lại accuracy vì tăng tốc có thể ảnh hưởng hộp nhỏ. E3 có lợi thế dung lượng; E4 cân bằng; E6 cần tăng tốc mạnh nếu nhắm thời gian thực trên CPU. Không suy đoán mức tăng từ tài liệu nhà cung cấp mà phải đo checkpoint ROD.

A.5 Giới hạn liên quan người dùng khiếm thị

Detector ảnh tĩnh chỉ trả tên và vị trí pixel. Nó không biết vật thể cách bao xa, có nằm trên quỹ đạo hay đang tiến lại gần. Cảnh báo mọi detection có thể gây quá tải; bỏ qua detection có thể gây rủi ro. Thiết kế cảnh báo cần nghiên cứu người dùng, ưu tiên theo nguy cơ, cooldown, tracking và thử nghiệm có giám sát.

Âm thanh phải ngắn, dễ hiểu và không che âm thanh môi trường. Phản hồi rung cần quy ước nhất quán. Các quyết định này không thể suy từ mAP. Trước thử nghiệm người dùng cần đánh giá an toàn, đồng thuận, quyền riêng tư hình ảnh và cơ chế dừng khẩn cấp. Ứng dụng hiện chỉ là demo nghiên cứu ảnh tải lên.

A.6 Ánh xạ 25 lớp

ROD ID	Torchvision ID	Tên lớp
0	1	Bike
1	2	Building
2	3	Car
3	4	Person
4	5	Stairs
5	6	Traffic sign
6	7	Electrical Pole
7	8	Road
8	9	Motorcycle
9	10	Dustbin
10	11	Dog
11	12	Manhole
12	13	Tree
13	14	Guard rail
14	15	Pedestrian crosswalk
15	16	Truck
16	17	Bus

ROD ID	Torchvision ID	Tên lớp
17	18	Bench
18	19	Traffic Cone
19	20	Fire hydrant
20	21	Teraffic Barrel
21	22	Plant Pot
22	23	Electrical Box
23	24	Chair
24	25	Bicycle Rack

Torchvision ID 0 là background và không ánh xạ thành lớp ROD. YOLO dùng trực tiếp ROD ID 0–24. Bảng phải được kiểm khi thay dataset hoặc checkpoint; sai một đơn vị có thể tạo hộp đúng nhưng tên lớp sai cho toàn bộ ứng dụng.